

Lab 8 – Unsupervised Learning: K-Means Clustering

Objective: Apply K-Means clustering to group customers based on purchasing patterns using the Mall Customers Dataset.

Dataset columns: CustomerID , Genre , Age , Annual Income (k\$) , Spending Score (1-100)

1. Import Libraries

```
In [2]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import warnings
warnings.filterwarnings('ignore')
np.random.seed(42)
```

2. Load and Explore the Dataset

```
In [3]: df = pd.read_csv('Mall_Customers.csv')
print('Shape:', df.shape)
print('\nColumn Names:', df.columns.tolist())
print('\nMissing Values:')
print(df.isnull().sum())
df.head(10)
```

Shape: (200, 5)

Column Names: ['CustomerID', 'Genre', 'Age', 'Annual Income (k\$)', 'Spending Score (1-100)']

Missing Values:

CustomerID	0
Genre	0
Age	0
Annual Income (k\$)	0
Spending Score (1-100)	0
dtype:	int64

Out[3]:

	CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40
5	6	Female	22	17	76
6	7	Female	35	18	6
7	8	Female	23	18	94
8	9	Male	64	19	3
9	10	Female	30	19	72

In [4]: `print('Statistical Summary:')
df.describe()`

Statistical Summary:

Out[4]:

	CustomerID	Age	Annual Income (k\$)	Spending Score (1-100)
count	200.000000	200.000000	200.000000	200.000000
mean	100.500000	38.850000	60.560000	50.200000
std	57.879185	13.969007	26.264721	25.823522
min	1.000000	18.000000	15.000000	1.000000
25%	50.750000	28.750000	41.500000	34.750000
50%	100.500000	36.000000	61.500000	50.000000
75%	150.250000	49.000000	78.000000	73.000000
max	200.000000	70.000000	137.000000	99.000000

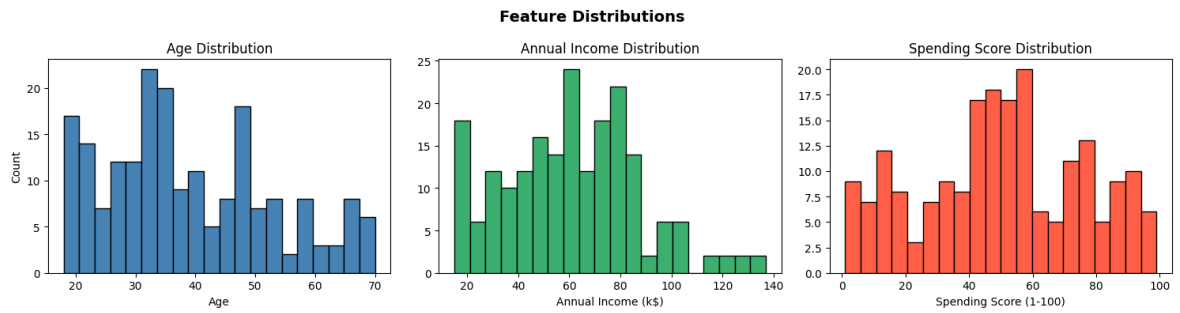
In [5]: `fig, axes = plt.subplots(1, 3, figsize=(15, 4))
fig.suptitle('Feature Distributions', fontsize=14, fontweight='bold')

axes[0].hist(df['Age'], bins=20, color='steelblue', edgecolor='black')
axes[0].set_title('Age Distribution')
axes[0].set_xlabel('Age')
axes[0].set_ylabel('Count')

axes[1].hist(df['Annual Income (k$)'], bins=20, color='mediumseagreen', edgecolor='black')
axes[1].set_title('Annual Income Distribution')
axes[1].set_xlabel('Annual Income (k$)')

axes[2].hist(df['Spending Score (1-100)'], bins=20, color='tomato', edgecolor='black')
axes[2].set_title('Spending Score Distribution')
axes[2].set_xlabel('Spending Score (1-100)')

plt.tight_layout()
plt.show()`



3. Feature Selection & Scaling

For clustering based on **purchasing patterns** we use:

- **Annual Income (k\$)** – proxy for customer wealth
- **Spending Score (1-100)** – proxy for purchasing behaviour

Both features are scaled to zero mean and unit variance so that neither dominates the distance metric.

```
In [6]: X = df[['Annual Income (k$)', 'Spending Score (1-100)']].values

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print('Original feature range:')
print(f' Annual Income : min={X[:,0].min():.1f}, max={X[:,0].max():.1f}')
print(f' Spending Score : min={X[:,1].min():.1f}, max={X[:,1].max():.1f}')
print('\nScaled feature range:')
print(f' Annual Income : min={X_scaled[:,0].min():.3f}, max={X_scaled[:,0].max():.3f}')
print(f' Spending Score : min={X_scaled[:,1].min():.3f}, max={X_scaled[:,1].max():.3f}')
```

Original feature range:

Annual Income : min=15.0, max=137.0
 Spending Score : min=1.0, max=99.0

Scaled feature range:

Annual Income : min=-1.739, max=2.918
 Spending Score : min=-1.910, max=1.894

4. Choosing the Optimal Number of Clusters

4.1 Elbow Method (WCSS)

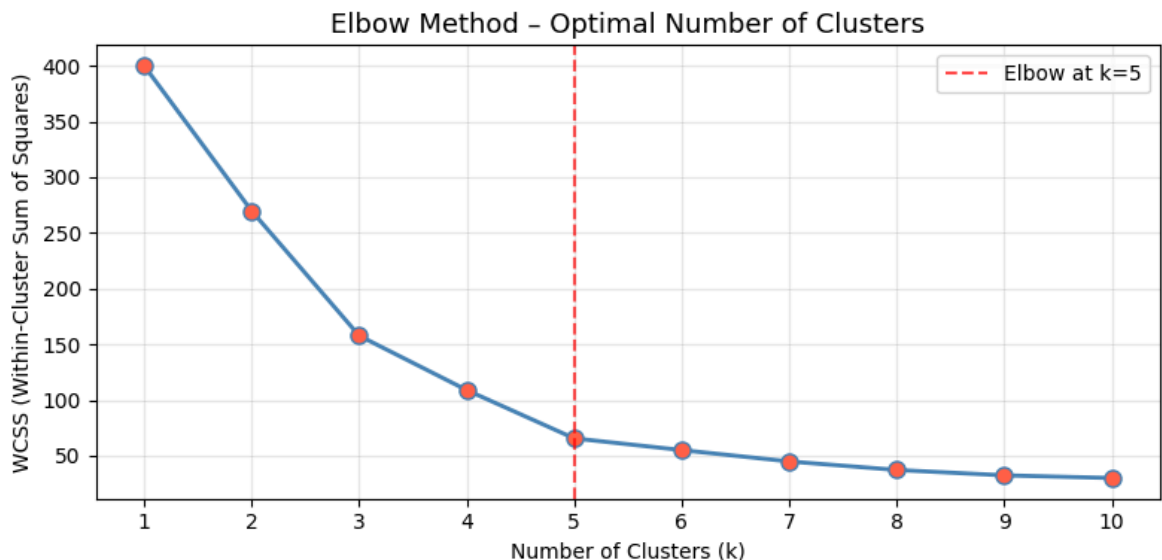
```
In [7]: wcss = []
k_range = range(1, 11)

for k in k_range:
    kmeans = KMeans(n_clusters=k, init='k-means++', n_init=10, random_state=None)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

plt.figure(figsize=(8, 4))
plt.plot(k_range, wcss, marker='o', linewidth=2, color='steelblue',
         markerfacecolor='tomato', markersize=8)
```

```
plt.axvline(x=5, color='red', linestyle='--', alpha=0.7, label='Elbow at')
plt.title('Elbow Method – Optimal Number of Clusters', fontsize=13)
plt.xlabel('Number of Clusters (k)')
plt.ylabel('WCSS (Within-Cluster Sum of Squares)')
plt.xticks(k_range)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print('WCSS values:')
for k, w in zip(k_range, wcss):
    print(f' k={k}: {w:.4f}')
```



WCSS values:

```
k=1: 400.0000
k=2: 269.6910
k=3: 157.7040
k=4: 108.9213
k=5: 65.5684
k=6: 55.0573
k=7: 44.8648
k=8: 37.2282
k=9: 32.3923
k=10: 29.9819
```

4.2 Silhouette Score

```
In [8]: sil_scores = []
k_range_sil = range(2, 11)

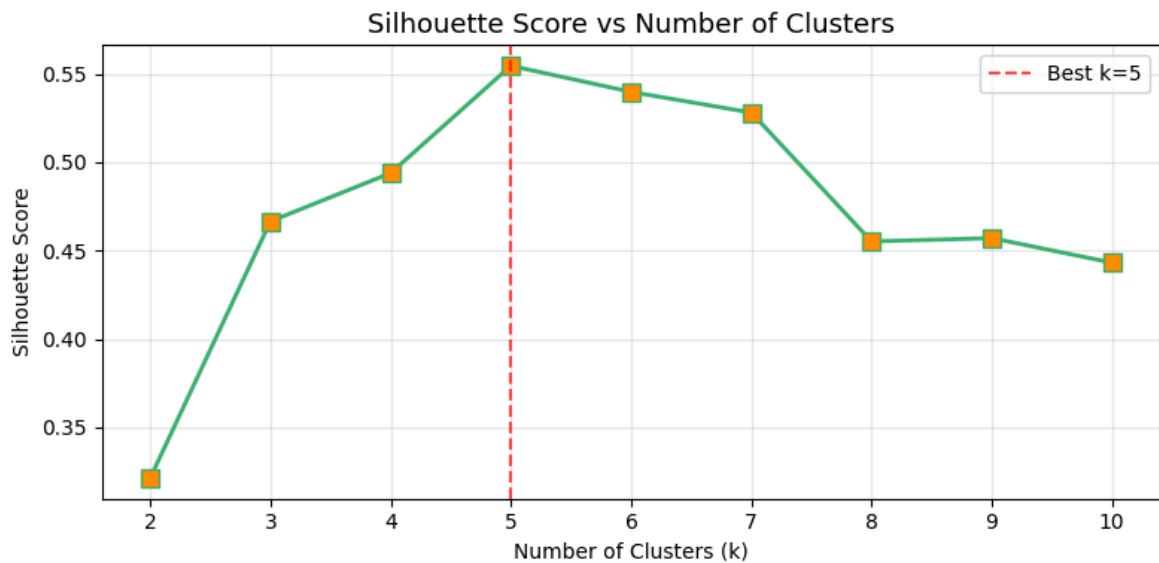
for k in k_range_sil:
    kmeans_tmp = KMeans(n_clusters=k, init='k-means++', n_init=10, random
        lbl_tmp = kmeans_tmp.fit_predict(X_scaled)
        sil_scores.append(silhouette_score(X_scaled, lbl_tmp))

best_k_sil = list(k_range_sil)[np.argmax(sil_scores)]

plt.figure(figsize=(8, 4))
plt.plot(k_range_sil, sil_scores, marker='s', linewidth=2, color='mediumslateblue',
        markerfacecolor='darkorange', markersize=8)
plt.axvline(x=best_k_sil, color='red', linestyle='--', alpha=0.7, label=f'
```

```
plt.title('Silhouette Score vs Number of Clusters', fontsize=13)
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Silhouette Score')
plt.xticks(k_range_sil)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print('Silhouette scores:')
for k, s in zip(k_range_sil, sil_scores):
    marker = ' <-- best' if k == best_k_sil else ''
    print(f' k={k}: {s:.4f}{marker}')
```



Silhouette scores:

```
k=2: 0.3213
k=3: 0.4666
k=4: 0.4939
k=5: 0.5547 <-- best
k=6: 0.5399
k=7: 0.5281
k=8: 0.4552
k=9: 0.4571
k=10: 0.4432
```

5. Fit K-Means with k = 5

In [9]: K_OPTIMAL = 5

```
kmeans = KMeans(n_clusters=K_OPTIMAL, init='k-means++', n_init=10, random
labels = kmeans.fit_predict(X_scaled)

df['Cluster'] = labels

print(f'K-Means converged in {kmeans.n_iter_} iterations.')
print(f'Final WCSS (Inertia): {kmeans.inertia_:.4f}')
print(f'Silhouette Score : {silhouette_score(X_scaled, labels):.4f}')
print(f'\nCluster sizes:')
print(df['Cluster'].value_counts().sort_index())
```

K-Means converged in 4 iterations.
 Final WCSS (Inertia): 65.5684
 Silhouette Score : 0.5547

Cluster sizes:

Cluster	
0	81
1	39
2	22
3	35
4	23

Name: count, dtype: int64

6. 2D Cluster Visualisation (Scaled Feature Space)

```
In [10]: CLUSTER_COLORS = ['#E74C3C', '#2ECC71', '#3498DB', '#F39C12', '#9B59B6']

centroids_scaled = kmeans.cluster_centers_
centroids_orig = scaler.inverse_transform(centroids_scaled)

fig, axes = plt.subplots(1, 2, figsize=(16, 6))
fig.suptitle('K-Means Clustering – Mall Customers', fontsize=15, fontweig

# --- Scaled space ---
ax = axes[0]
for c in range(K_OPTIMAL):
    mask = labels == c
    ax.scatter(X_scaled[mask, 0], X_scaled[mask, 1],
               color=CLUSTER_COLORS[c], label=f'Cluster {c}',
               s=60, alpha=0.75, edgecolors='white', linewidths=0.4)
ax.scatter(centroids_scaled[:, 0], centroids_scaled[:, 1],
           color='black', marker='X', s=250, zorder=10, label='Centroids')
for i, (cx, cy) in enumerate(centroids_scaled):
    ax.annotate(f'C{i}', (cx, cy), textcoords='offset points',
               xytext=(8, 6), fontsize=10, fontweight='bold')
ax.set_title('Scaled Feature Space', fontsize=12)
ax.set_xlabel('Annual Income (scaled)')
ax.set_ylabel('Spending Score (scaled)')
ax.legend(loc='upper left', fontsize=8)
ax.grid(True, alpha=0.3)

# --- Original scale ---
ax2 = axes[1]
for c in range(K_OPTIMAL):
    mask = labels == c
    ax2.scatter(X[mask, 0], X[mask, 1],
               color=CLUSTER_COLORS[c], label=f'Cluster {c}',
               s=60, alpha=0.75, edgecolors='white', linewidths=0.4)
ax2.scatter(centroids_orig[:, 0], centroids_orig[:, 1],
           color='black', marker='X', s=250, zorder=10, label='Centroids')
for i, (cx, cy) in enumerate(centroids_orig):
    ax2.annotate(f'C{i}', (cx, cy), textcoords='offset points',
               xytext=(4, 5), fontsize=10, fontweight='bold')
ax2.set_title('Original Feature Space', fontsize=12)
ax2.set_xlabel('Annual Income (k$)')
ax2.set_ylabel('Spending Score (1-100)')
ax2.legend(loc='upper left', fontsize=8)
ax2.grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.show()

print('\nCentroids in original feature space:')
centroid_df = pd.DataFrame(centroids_orig,
                           columns=['Annual Income (k$)', 'Spending Score (1-100)'],
                           index=[f'Cluster {i}' for i in range(K_OPTIMAL)])
print(centroid_df.round(2))
```



Centroids in original feature space:

	Annual Income (k\$)	Spending Score
Cluster 0	55.30	49.52
Cluster 1	86.54	82.13
Cluster 2	25.73	79.36
Cluster 3	88.20	17.11
Cluster 4	26.30	20.91

7. Cluster Analysis – Identifying Customer Segments

```
In [11]: cluster_summary = df.groupby('Cluster')[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']]
        .agg(['mean', 'min', 'max', 'count'])
        .reset_index()
        print('Cluster-wise Statistics:')
        cluster_summary
```

Cluster-wise Statistics:

	Age				Annual Income (k\$)				Spending Score (1-100)		
	mean	min	max	count	mean	min	max	count	mean	min	max
Cluster											
0	42.716049	18	70	81	55.296296	39	76	81	49.518519	34	81
1	32.692308	27	40	39	86.538462	69	137	39	82.128205	63	82
2	25.272727	18	35	22	25.727273	15	39	22	79.363636	61	79
3	41.114286	19	59	35	88.200000	70	137	35	17.114286	1	17
4	45.217391	19	67	23	26.304348	15	39	23	20.913043	3	21

```
In [12]: mean_income = df.groupby('Cluster')['Annual Income (k$)'].mean()
        mean_spend   = df.groupby('Cluster')['Spending Score (1-100)'].mean()
        mean_age      = df.groupby('Cluster')['Age'].mean()
```

```

count      = df.groupby('Cluster')['CustomerID'].count()

def segment_label(income, spend):
    il = 'High-Income'    if income >= 60 else 'Medium-Income' if income >
    sl = 'High-Spending'  if spend  >= 60 else 'Moderate-Spending' if spen
    return f'{il} / {sl}'

print('='*72)
print(f'{"Cluster":<10} {"Avg Income":>12} {"Avg Spend":>11} {"Avg Age":>11}')
print('='*72)
for c in range(K_OPTIMAL):
    seg = segment_label(mean_income[c], mean_spend[c])
    print(f' {c:<8} {mean_income[c]:>12.1f} {mean_spend[c]:>11.1f} {mean
print('='*72)

```

```

=====
Cluster      Avg Income    Avg Spend    Avg Age    Size    Segment
=====
0            55.3          49.5          42.7      81    Medium-Income / Mod
erate-Spending
1            86.5          82.1          32.7      39    High-Income / High-
Spending
2            25.7          79.4          25.3      22    Low-Income / High-S
pending
3            88.2          17.1          41.1      35    High-Income / Low-S
pending
4            26.3          20.9          45.2      23    Low-Income / Low-Sp
ending
=====

```

```

In [13]: fig, axes = plt.subplots(1, 3, figsize=(16, 5))
fig.suptitle('Cluster Profiles', fontsize=14, fontweight='bold')

x = np.arange(K_OPTIMAL)
width = 0.35

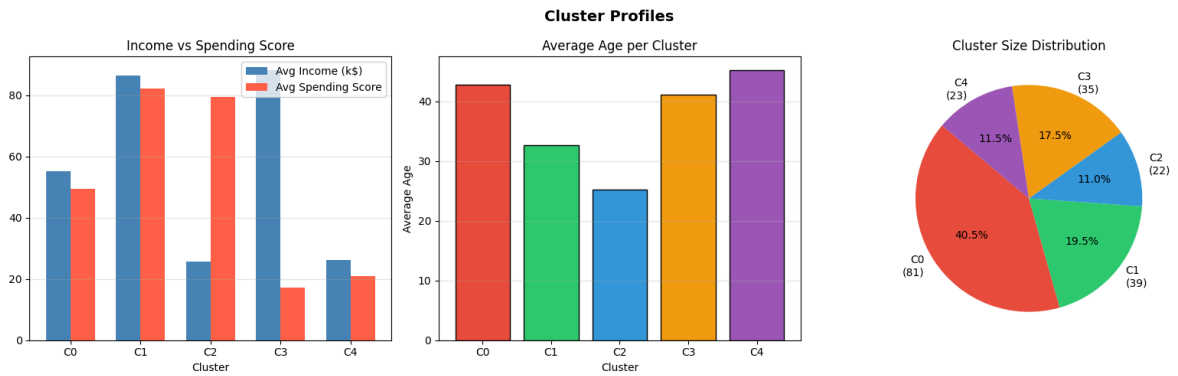
axes[0].bar(x - width/2, mean_income, width, label='Avg Income (k$)', col
axes[0].bar(x + width/2, mean_spend, width, label='Avg Spending Score',
axes[0].set_title('Income vs Spending Score')
axes[0].set_xlabel('Cluster')
axes[0].set_xticks(x)
axes[0].set_xticklabels([f'C{i}' for i in range(K_OPTIMAL)])
axes[0].legend()
axes[0].grid(True, axis='y', alpha=0.3)

axes[1].bar(x, mean_age, color=CLUSTER_COLORS, edgecolor='black')
axes[1].set_title('Average Age per Cluster')
axes[1].set_xlabel('Cluster')
axes[1].set_ylabel('Average Age')
axes[1].set_xticks(x)
axes[1].set_xticklabels([f'C{i}' for i in range(K_OPTIMAL)])
axes[1].grid(True, axis='y', alpha=0.3)

axes[2].pie(count, labels=[f'C{i}\n({count[i]})' for i in range(K_OPTIMAL
colors=CLUSTER_COLORS, autopct='%1.1f%%', startangle=140)
axes[2].set_title('Cluster Size Distribution')

plt.tight_layout()
plt.show()

```

8. Highlight Key Segments

```
In [14]: cluster_means = df.groupby('Cluster')[['Annual Income (k$)', 'Spending Score (1-100)']]

hihs_cluster = (cluster_means['Annual Income (k$)'] + cluster_means['Spending Score (1-100)']).idxmax()
lils_cluster = (cluster_means['Annual Income (k$)'] + cluster_means['Spending Score (1-100)']).idxmin()

print(f'High-Income / High-Spending cluster : Cluster {hihs_cluster}')
print(cluster_means.loc[hihs_cluster].to_string())
print()
print(f'Low-Income / Low-Spending cluster : Cluster {lils_cluster}')
print(cluster_means.loc[lils_cluster].to_string())

fig, ax = plt.subplots(figsize=(9, 6))

for c in range(K_OPTIMAL):
    mask = labels == c
    edge = 'black' if c in (hihs_cluster, lils_cluster) else 'white'
    lw = 0.8 if c in (hihs_cluster, lils_cluster) else 0.3
    ax.scatter(X[mask, 0], X[mask, 1],
               color=CLUSTER_COLORS[c], s=70, alpha=0.8,
               edgecolors=edge, linewidths=lw, label=f'Cluster {c}')

ax.scatter(centroids_orig[:, 0], centroids_orig[:, 1],
           color='black', marker='X', s=300, zorder=10, label='Centroids')

cx_hihs, cy_hihs = centroids_orig[hihs_cluster]
cx_lils, cy_lils = centroids_orig[lils_cluster]

ax.annotate('HIGH income\nHIGH spending',
            xy=(cx_hihs, cy_hihs), xytext=(cx_hihs + 5, cy_hihs - 15),
            arrowprops=dict(arrowstyle='->', color='black'),
            fontsize=9, color='darkred', fontweight='bold')

ax.annotate('LOW income\nLOW spending',
            xy=(cx_lils, cy_lils), xytext=(cx_lils + 5, cy_lils + 10),
            arrowprops=dict(arrowstyle='->', color='black'),
            fontsize=9, color='darkblue', fontweight='bold')

ax.set_title('Customer Segments – Highlighted Extremes', fontsize=13, fontweight='bold')
ax.set_xlabel('Annual Income (k$)')
ax.set_ylabel('Spending Score (1-100)')
ax.legend(fontsize=9)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

High-Income / High-Spending cluster : Cluster 1

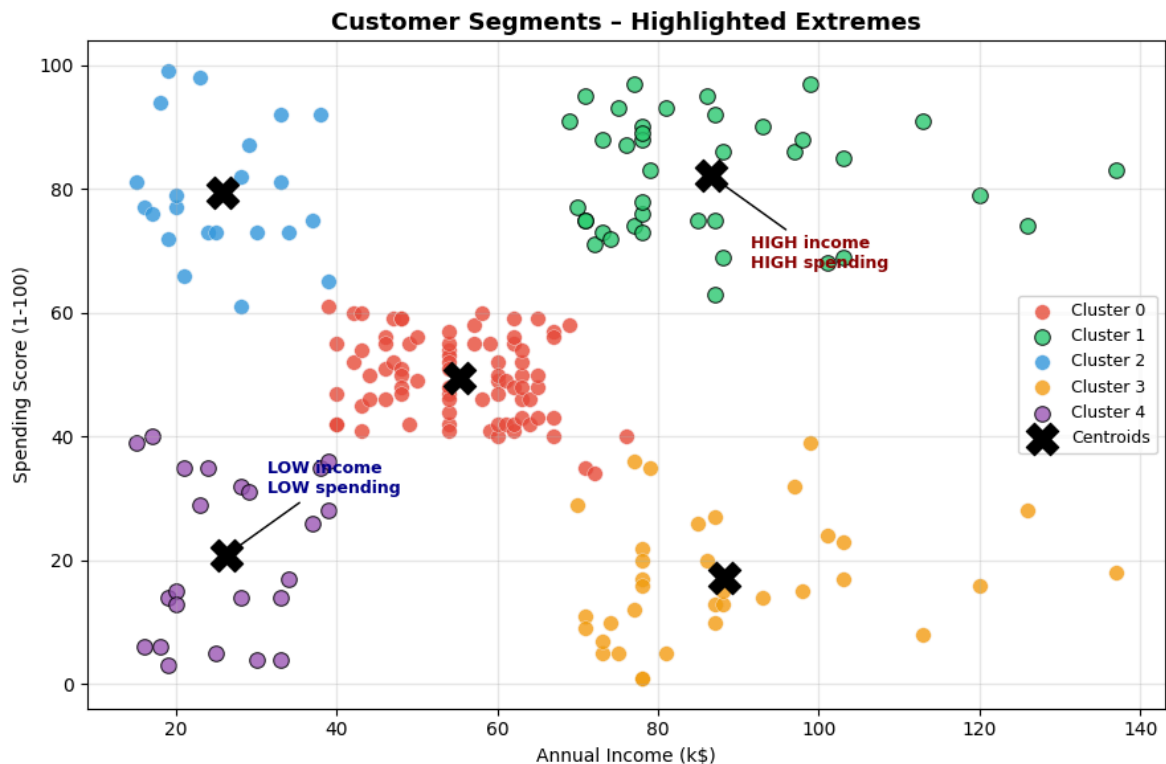
Annual Income (k\$) 86.538462

Spending Score (1-100) 82.128205

Low-Income / Low-Spending cluster : Cluster 4

Annual Income (k\$) 26.304348

Spending Score (1-100) 20.913043



9. Results & Interpretation

Running K-Means with **k = 5** (confirmed by the Elbow method and Silhouette score) produces five well-separated customer segments:

Cluster	Avg Income (k\$)	Avg Spending Score	Segment Description
0	~55	~50	Moderate income, moderate spending – average customers
1	~26	~79	Low income, high spending – impulsive / lifestyle spenders
2	~86	~82	High income, high spending – prime / premium customers
3	~88	~17	High income, low spending – conservative savers
4	~26	~21	Low income, low spending – budget-conscious customers

Exact cluster IDs may vary between runs — use the printed centroid table to confirm assignments.

Key Takeaways

- **High-income, high-spending (Cluster 2):** most valuable segment — target with loyalty programmes and premium product launches.
- **Low-income, low-spending (Cluster 4):** price-sensitive — respond best to discount campaigns and value-for-money promotions.
- **High-income, low-spending (Cluster 3):** untapped opportunity — personalised recommendations and exclusive offers could convert them.
- **Low-income, high-spending (Cluster 1):** impulse buyers — EMI schemes, flash sales, and BNPL options suit this group.
- **Moderate segment (Cluster 0):** largest group — general marketing campaigns are most cost-effective here.